
Unit – 4

I. Code generation

Issues in the design of a code Generator, Basic blocks and flow graphs, Next-use information, A simple Code generator, DAG representation of Basic blocks, Peephole Optimization, Generating code from DAGS.

II. Code optimization

The principle sources of optimization, Optimization of basic blocks, Implementation for Common Sub expression technique using DAG.

III. Symbol table: The contents of a symbol table, Data structures for Symbol Table, Representing scope information.

I. CODE GENERATION

- The final phase in compiler model is the code generator.
- It takes as input an intermediate representation of the source program and produces as output an equivalent target program.

1. Issues in the Design of a Code Generator

- The design of the code generator is dependent on the specifics of the intermediate representation, the target language and the run-time systems such as,
 1. **Input to the code generator**
 2. **Target program**
 3. **Memory management**
 4. **Instruction selection**
 5. **Register allocation**
 6. **Evaluation order**

(i) Input to the code generator

- The input to the code generation consists of the intermediate representation of the source program produced by front end, together with information in the symbol table to determine run-time addresses of the data objects denoted by the names in the intermediate representation.
- The many choices for the Intermediate representations include,
 1. **Linear representation** such as postfix notation
 2. **Three address representation** such as quadruples, triples or indirect triples
 3. **Virtual machine representation** such as stack machine code
 4. **Graphical representations** such as syntax trees and DAG.
- Prior to code generation, the front end must be scanned, parsed and translated into intermediate representation along with necessary type checking. Therefore, input to code generation is assumed to be error-free.

(ii) The Target program

- The Instruction Set Architecture (ISA) of the target machine has a significant impact on the difficulty of designing a good code generator.
- The most common target-machine architectures are RISC, CISC and Stack based.
- The output of the code generator is the target program. The output may be

1. **Absolute machine language** - It can be placed in a fixed memory location and can be executed immediately.
2. **Relocatable machine language** - It allows subprograms to be compiled separately.
3. **Assembly language** - Code generation is made easier.

(iii) Memory management

- Names in the source program are mapped to addresses of data objects in run-time memory by the front end and code generator.
- It makes use of symbol table, that is, a name in a three-address statement refers to a symbol-table entry for the name.
- Labels in three-address statements have to be converted to addresses of instructions.

For example,

j: goto i where i is a label

generates jump instruction as follows,

if $i < j$, a **backward jump** instruction with target address equal to location of code for quadruple i is generated.

if $i > j$, the jump is forward. We must store on a list for quadruple i the location of the first machine instruction generated for quadruple j . When i is processed, the machine locations for all instructions that forward jumps to i are filled.

(iv) Instruction Selection

- The instructions of target machine should be complete and uniform.
- Instruction speeds and machine idioms are important factors when efficiency of target program is considered.
- The quality of the generated code is determined by its speed and size.

(v) Register allocation

- Instructions involving register operands are shorter and faster than those involving operands in memory.
- The use of registers is subdivided into two sub problems,
 1. **Register allocation** – the set of variables that will reside in registers at a point in the program is selected.
 2. **Register assignment** – the specific register that a variable will reside in is picked.
- Certain machine requires even-odd register pairs for some operands and results.

(vi) Evaluation order

- The order in which the computations are performed can affect the efficiency of the target code.
- Some computation orders require fewer registers to hold intermediate results than others.

2. Basic Blocks and Flow Graphs

Basic Blocks

- A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without any halt or possibility of branching except at the end.

Basic Block Construction Algorithm

Algorithm : Partition into basic blocks

Input : A sequence of three-address statements

Output : A list of basic blocks with each three-address statement in exactly one block

Steps :

1. First determine the set of leaders, the first statements of basic blocks.
The rules for determining the Leaders are,
 - a. The **first statement** is a leader.
 - b. Any statement that is the **target of a conditional or unconditional goto** is a leader.
 - c. Any statement that **immediately follows a conditional or unconditional goto** statement is a leader.
2. For each leader, its basic block consists of the leader and all statements up to but not including the next leader or the end of the program.

Flow Graphs

- Flow graph is a directed graph containing the flow-of-control information for the set of basic blocks making up a program.

Properties of Flow-graph

- The nodes of the flow graph are basic blocks and the first block is designated as a Initial block
- There is an directed edge from the basic block B1 to B2, iff B1 is a predecessor of B2.

Rules for determining the predecessor node

The block B1 is said to be predecessor to Block B2,

1. If B2 can immediately follows B1 in execution sequence or
2. If B2 immediately follows B1 in the execution sequence and B1 does not end with an unconditional goto statement.

Example:

For the following segment of the Code,

Begin

prod :=0;

i:=1;

do begin

prod :=prod+ a[i] * b[i];

i :=i+1;

```

while (i <= 20)
end

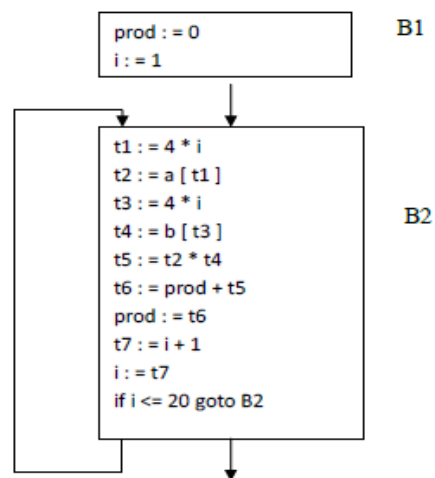
```

- The Three address code Instructions are,

```

(1)  prod := 0
(2)  i := 1
(3)  t1 := 4 * i
(4)  t2 := a[t1]
(5)  t3 := 4 * i
(6)  t4 := b[t3]
(7)  t5 := t2 * t4
(8)  t6 := prod + t5
(9)  prod := t6
(10) t7 := i + 1
(11) i := t7
(12) if i <= 20 goto (3)

```



- Leaders are statement No: (1) and (3).
Hence the Basic block B1: Statement (1) to (2) & Basic block B2: Statement (3) to (12).

3. Next-Use Information

- If the name in a register is no longer needed, then we remove the name from the register and the register can be used to store some other names.
- Next use information is used in code generation and register allocation.

The use of a name in a three-address statement is defined as

Let the TAC statement "i" assigns a value to the element x,

- If TAC statement "j" has x as an operand and control can flow from statement "i" to "j" along the path there is no intervening assignments to x, then the statement "j" uses the value of x computed at statement "i".
- The Next use Computation algorithm assumes that on exit from the basic block,
 - All temporaries are considered non-live
 - All variables defined in the source program (and non-temps) are live
 - Each procedure/function call is assumed to start a basic block

Algorithm to determine Liveliness and Next-use information

Input : Basic block B of three-address statements

Output : At each statement i: x= y op z, we attach to i the liveliness and next-uses of x, y and z.

Method : We start at the last statement of B and scan backwards.

- Attach to statement i the information currently found in the symbol table regarding the next-use and liveliness of x, y and z.
- In the symbol table, set x to "not live" and "no next use".
- In the symbol table, set y and z to "live", and next-uses of y and z to i.

Example: Consider the following TAC sequence in a Basic Block

[NNU – No Next Use; NU – Next Use, NLive – Not Live]

(1)	prod := 0	{prod- [Live, NNU] }
(2)	i := 1	{ i – [Live, NU – (3)] }
(3)	t1 := 4* i	{ t1 – [NLive, NU – (4), i-[Live, NNU]] }
(4)	t2 := a[t1]	{ t2 – [NLive, NNU], t1 – [NLive, NNU], a – [Live, NNU] }

4. The Simple Code Generator – Algorithm

- A code generator generates target code for a sequence of three-address statements in a basic block and effectively uses registers to store operands of the statements.
- It keeps track of what values are in what register so it can avoid generating unnecessary load and stores.
- This algorithm assumes that
 - The basic block has already been optimized.
 - For each operator, there is exactly one machine instruction that takes necessary operands in registers and performs that operation.

Data structures**1. Register Descriptor**

- A Register Descriptor keeps track of the variables whose current values are in that Register.
- Initially all register descriptors are empty.

2. Address Descriptor

- An Address descriptor keeps track of the location or locations where the current value of that variable can be found.
- The location can be a register, memory location, a stack location or some set of more than one of these.

3. getReg () function

- This function selects registers for each memory location associated with the TAC statement
- It has access to the register and address descriptors for all variables of the basic block.

Machine Instructions for Computation OperationsFor each three-address statement of the form **x := y op z**, perform the following actions,

1. Use **getReg (x := y op z)** to select registers for x, y and z. Call these as Rx, Ry and Rz
2. If y is not in Ry (according to register descriptor for Ry), then
 Issue an instruction LD Ry, y'
 where y' is one of the memory locations for y (according to address descriptor for y).
3. Similarly if z is not in Rz, then
 Issue an instruction LD Rz, z'
 where z' is one of the memory locations for z (according to address descriptor for z).
4. **Issue the instruction ADD Rx, Ry, Rz**

Machine Instructions for Copy statements

The three-address copy statement of the form $x = y$, the `getReg ()` function always assumes choose the same register for both x and y .

1. If y is not already in register R_y , then

Issue an instruction LD R_y, y

Else do nothing.

2. Adjust the register descriptor for R_y , so that it includes x as one of the value found.

Ending the Basic Block

- For each user defined variable (assumed to be Live on Exit) x whose address descriptor does not say that its value is located in the memory location for x , then

Generate the instruction ST R, x

where R is the register in which x 's value exists at the end of the block.

Example: The assignment $d := (a-b) + (a-c) + (a-c)$ might be translated into the following three - address code sequence as [No of registers available: $R1, R2, R3$]

$t = a - b$

$u = a - c$

$v = t + u$

$d = v + u$

Reg. Descriptor			Address Descriptors						
R1	R2	R3	a	b	c	d	t	u	v
			a	b	c	d			

$t = a - b$

LD $R1, a$

LD $R2, b$

SUB $R2, R1, R2$

a	t		a,R1	b	c	d	R2		
---	---	--	------	---	---	---	----	--	--

$u = a - c$

LD $R3, c$

SUB $R1, R1, R3$

u	t	c	a	b	C, R3	d	R2	R1	
---	---	---	---	---	-------	---	----	----	--

$v = t + u$

ADD $R2, R2, R1$

u	v	c	a	b	c, R3	d		R1	R2
---	---	---	---	---	-------	---	--	----	----

$d = v + u$

ADD $R1, R1, R2$

d	v	c	a	b	c, R3	d,R1			R2
---	---	---	---	---	-------	------	--	--	----

5. The DAG Representation for Basic Blocks

- A DAG for a basic block is a directed acyclic graph with the following labels on nodes
 1. Leaves are labeled by unique identifiers, either variable names or constants.
 2. Interior nodes are labeled by an operator symbol.
 3. Nodes are also optionally given a sequence of identifiers for labels to store the computed values.
- DAGs are useful data structures for implementing transformations on basic blocks.
- It gives a picture of how the value computed by a statement is used in subsequent statements.
- It provides a good way of determining common sub - expressions.

Algorithm of Construction of DAG

Input : A basic block

Output : A DAG for the basic block containing the following information:

1. A label for each node.
 - For leaves, the label is an identifier.
 - For interior nodes, an operator symbol.
2. For each node, a list of attached identifiers to hold the computed values.
 - Case (i) $x := y \text{ OP } z$
 - Case (ii) $x := \text{OP } y$
 - Case (iii) $x := y$

Method:

Step 1: If y is undefined then create node(y). If z is undefined, create node (z) for case(i).

Step 2:

For the Case (i):

Create a node (OP) whose left child is node(y) and right child is node (z). (Checking for common sub expression. Let n be this node)

For Case (ii):

Determine whether there is node (OP) with one child node(y). If not create such a node.

For Case (iii):

Node n will be node(y).

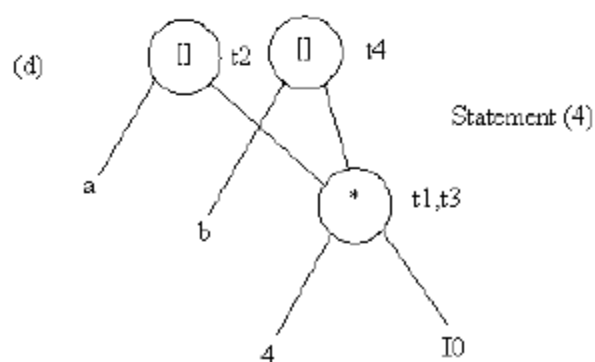
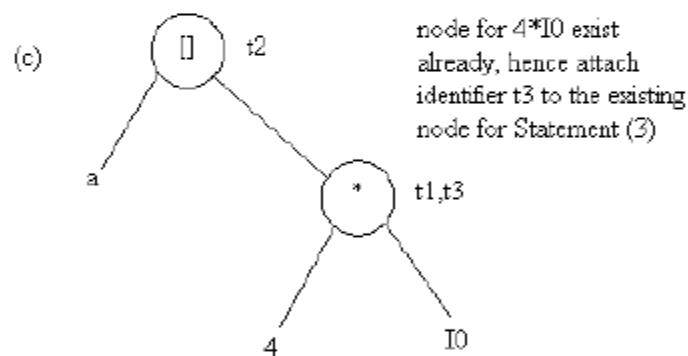
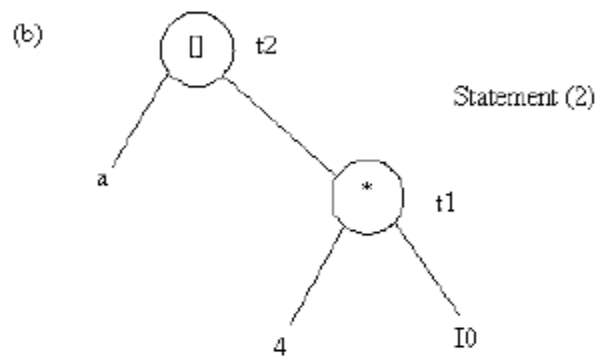
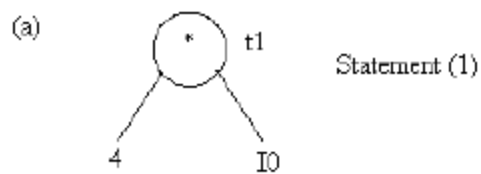
Step 3: Delete x from the list of identifiers for node(x). Append x to the list of attached identifiers for the node n found in step 2 and set node(x) to n.

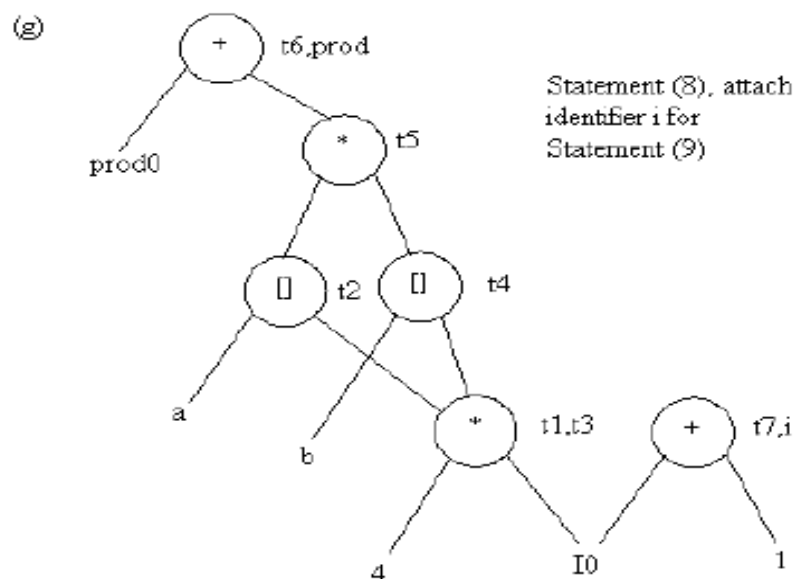
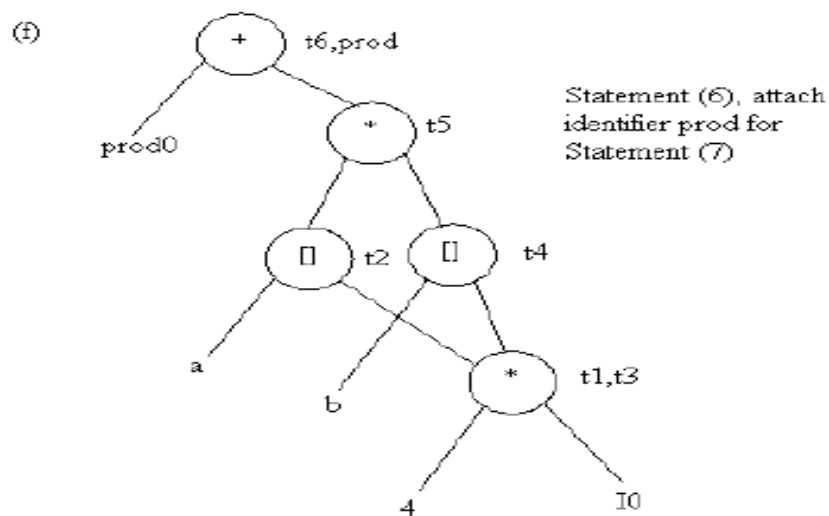
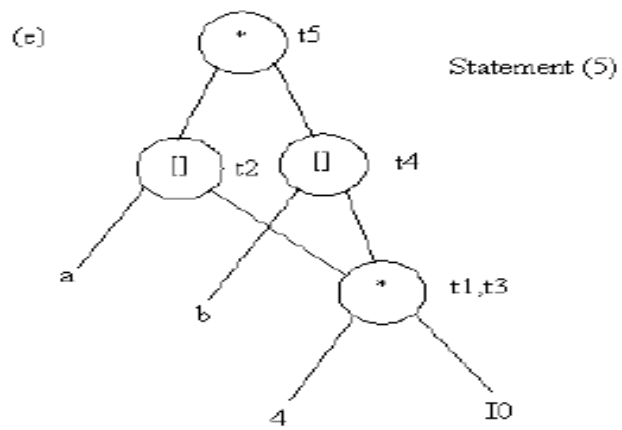
Example: Construct the DAG representation for the given basic block →

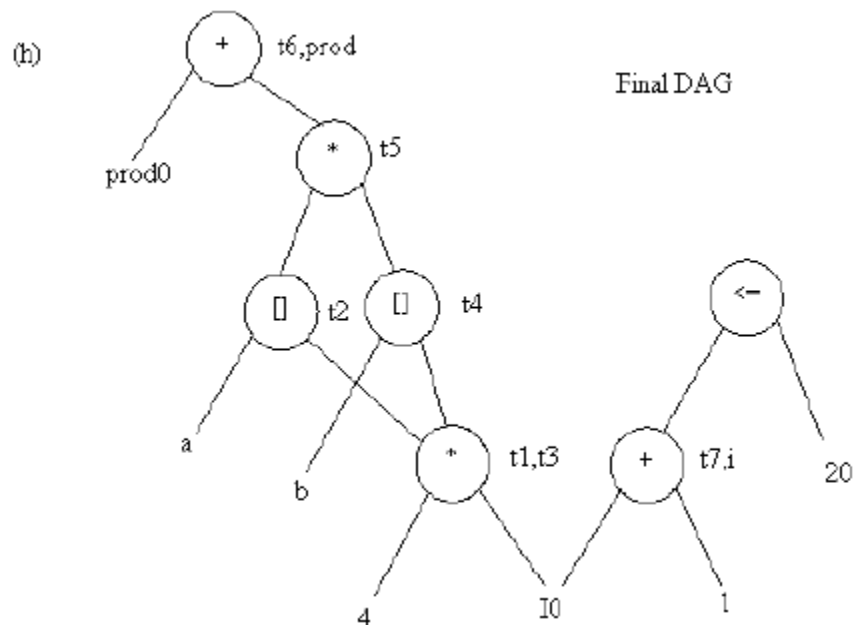
```

1.  $t_1 := 4 * i$ 
2.  $t_2 := a[t_1]$ 
3.  $t_3 := 4 * i$ 
4.  $t_4 := b[t_3]$ 
5.  $t_5 := t_2 * t_4$ 
6.  $t_6 := \text{prod} + t_5$ 
7.  $\text{prod} := t_6$ 
8.  $t_7 := i + 1$ 
9.  $i := t_7$ 
10. if  $i \leq 20$  goto (1)
  
```

Stages in DAG Construction







Application of DAG

1. We can automatically detect common sub expressions.
2. We can determine which identifiers have their values used in the block.
3. We can determine which statements compute values that could be used outside the block.

6. Peephole Optimization

- A statement-by-statement code-generations strategy often produce target code that contains redundant instructions and suboptimal constructs .The quality of such target code can be improved by applying “optimizing” transformations to the target program.
- A simple but effective technique for improving the target code is peephole optimization, a method for trying to improve the performance of the target program by examining a short sequence of target instructions (called the peephole) and replacing these instructions by a shorter or faster sequence, whenever possible.
- The peephole is a small, moving window on the target program. The code in the peephole need not contiguous, although some implementations do require this. It is characteristic of peephole optimization that each improvement may spawn opportunities for additional improvements.
- **Transformations of peephole optimizations**
 1. Redundant-instructions elimination
 2. Flow-of-control optimizations
 3. Algebraic simplifications
 4. Use of machine idioms
 5. Unreachable Code

(i) Redundant Loads and Stores

If we see the instructions sequence

(1) MOV R0, a

(2) MOV a, R0

We can delete instructions (2) because whenever (2) is executed. (1) will ensure that the value of a is already in register R0. If (2) had a label we could not be sure that (1) was always executed immediately before (2) and so we could not remove (2).

(ii) Unreachable Code

- Another opportunity for peephole optimizations is the removal of unreachable instructions.
- An unlabeled instruction immediately following an unconditional jump may be removed.
- This operation can be repeated to eliminate a sequence of instructions.
- For example, for debugging purposes, a large program may have within it certain segments that are executed only if a variable debug is 1. In C, the source code might look like:

```
#define debug 0
....
If ( debug ) {
    Print debugging information
}
```

In the intermediate representations the if-statement may be translated as:

```
If debug =1 goto L2
goto L2
L1: print debugging information
L2: .....(a)
```

- One obvious **peephole optimization is to eliminate jumps over jumps**. Thus no matter what the value of debug; (a) can be replaced by:


```
If debug ≠1 goto L2
Print debugging information
L2: .....(b)
```
- As the argument of the statement of (b) evaluates to a constant true it can be replaced by


```
If debug ≠0 goto L2
Print debugging information
L2: .....(c)
```
- As the argument of the first statement of (c) evaluates to a constant true, it can be replaced by goto L2. Then all the statement that print debugging aids are manifestly unreachable and can be eliminated one at a time.

(iii) Flows of Control Optimizations

- The unnecessary jumps can be eliminated in either the intermediate code or the target code by the following types of peephole optimizations. We can replace the jump sequence,

goto L1		goto L2
....	by the sequence	
L1: goto L2		L1: goto L2

- If there are now no jumps to L1, then it may be possible to eliminate the statement L1: goto L2 provided it is preceded by an unconditional jump.

- Similarly, the sequence

if a < b goto L1		If a < b goto L2
....	can be replaced by	
L1: goto L2		L1: goto L2

- Finally, suppose there is only one jump to L1 and L1 is preceded by an unconditional goto. Then the sequence

goto L1		If a < b goto L2
.....	May be replaced by	goto L3
L1: if a < b goto L2		
L3:(1)		L3:(2)

- While the number of instructions in (1) and (2) is the same, we sometimes skip the unconditional jump in (2), but never in (1). Thus (2) is superior to (1) in execution time

(iv) Algebraic Simplification

- There is no end to the amount of algebraic simplification that can be attempted through peephole optimization.
- Only a few algebraic identities occur frequently enough that it is worth considering implementing them. For example, statements such as

$x := x + 0$ or $x := x * 1$

- Are often produced by straightforward intermediate code-generation algorithms, and they can be eliminated easily through peephole optimization.

(v) Reduction in Strength

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine. Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators.
- For example, x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine.

(vi) Use of Machine Idioms

- The target machine may have hardware instructions to implement certain specific operations efficiently. For example, some machines have auto-increment and auto-decrement addressing modes.

7. Generating Code from DAGs

- The advantage of generating code for a basic block from its DAG representation is that, from a DAG we can easily see how to rearrange the order of the final computation sequence then we can start from a linear sequence of three-address statements or quadruples.

Rearranging the order

- The order in which computations are done can affect the cost of resulting object code. For example, consider the following basic block,

```

t1 := a + b
t2 := c + d
t3 := e - t2
t4 := t1 - t3

```

Generated code sequence for basic block,

```

MOV a , R0
ADD b , R0
MOV c , R1
ADD d , R1
MOV R0 , t1
MOV e , R0
SUB R1 , R0
MOV t1 , R1
SUB R0 , R1
MOV R1 , t4

```

Rearranged basic block,

Now t1 occurs immediately before t4.

```

t2 := c + d
t3 := e - t2
t1 := a + b
t4 := t1 - t3

```

Revised code sequence,

```

MOV c , R0
ADD d , R0
MOV a , R0
SUB R0 , R1
MOV a , R0
ADD b , R0
SUB R1 , R0
MOV R0 , t4

```

In this order, two instructions MOV R0, t1 and MOV t1, R1 have been saved.

II. CODE OPTIMIZATION

- Optimization is the process of transforming a piece of code to make more efficient (either in terms of time or space) without changing its output or side-effects.
- The only difference visible to the code's user should be that it runs faster and/or consumes less memory.
- Optimizations are classified into two categories. They are
 1. **Machine independent** optimizations
 2. **Machine dependant** optimizations

Machine independent optimizations - Machine independent optimizations are program transformations that improve the target code without taking into consideration any properties of the target machine.

Machine dependant optimizations - Machine dependant optimizations are based on register allocation and utilization of special machine-instruction sequences.

The criteria for code improvement transformations

1. Simply stated, the best program transformations are those that yield the most benefit for the least effort.
2. The transformation must preserve the meaning of programs. That is, the optimization must not change the output produced by a program for a given input, or cause an error that was not present in the original source program.
3. A transformation must, on the average, speed up programs by a measurable amount.
4. The transformation must be worth the effort. It does not make sense for a compiler writer to expend the intellectual effort, to implement a code improving transformation and to have the compiler expend the additional time on compiling source programs if this effort is not repaid when the target programs are executed.

1. THE PRINCIPLE SOURCES OF OPTIMIZATION

- A transformation of a program is called local if it can be performed by looking only at the statements in a basic block; otherwise, it is called global.
- Many transformations can be performed at both the local and global levels. Local transformations are usually performed first.

Function-Preserving Transformations / Semantics Preserving Transformations

- There are a number of ways in which a compiler can improve a program without changing the function it computes. The transformations are
 1. **Common sub expression elimination,**
 2. **Copy propagation,**
 3. **Dead-code elimination**
 4. **Constant folding**

(i) Common Sub expressions elimination

- An occurrence of an expression E is called a common sub-expression, if E was previously computed and the values of variables in E have not changed since the previous computation.
- We can avoid re-computing the expression if we can use the previously computed value.

Example

```

t1: = 4*i
t2: = a [t1]
t3: = 4*j
t4: = 4*i
t5: = n
t6: = b [t4] + t5

```

The common sub expression
t4: =4*i is eliminated
as its computation is already in t1
and the value of
"i" is not been changed from
definition to use.

```

t1: = 4*i
t2: = a [t1]
t3: = 4*j
t5: = n
t6: = b [t1] + t5

```

(ii) Copy Propagation

- Assignments of the form $f = g$ called copy statements, or copies for short.
- The idea behind the copy-propagation transformation is to use g for f , whenever possible after the copy statement $f = g$.
- Copy propagation means use of one variable instead of another. This may not appear to be an improvement, but as we shall see it gives us an opportunity to eliminate some element.

Example

```

x = pi;

```

```

.....

```

```

A = x*r*r;

```

The optimization using copy propagation can be done as follows:

```

A = pi*r*r;

```

Here the variable x is eliminated.

(iii) Dead-Code Eliminations - A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point.

- A related idea is dead or useless code, statements that compute values that never get used.
- While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations. An optimization can be done by eliminating dead code.

Example

```

i = 0;
if (i=1)
{
a=b+5;
}

```

Here, 'if' statement is dead code because this condition will never get satisfied. We can eliminate both the test and printing from the object code.

(iv) Constant folding

- More generally, deducing at compile time that the value of an expression is a constant and using the constant instead is known as constant folding.
- One advantage of copy propagation is that it often turns the copy statement into dead code.

Example

$a = 3.14 / 2;$
 can be replaced by
 $a = 1.57;$ thereby eliminating a division operation.

2. OPTIMIZATION OF BASIC BLOCKS

- There are two types of basic block optimizations. They are,
 1. **Structure-Preserving Transformations**
 2. **Algebraic Transformations**

Structure-Preserving Transformations

- The primary Structure-Preserving Transformation on basic blocks are
 1. **Common sub-expression elimination**
 2. **Dead code elimination**
 3. **Renaming of temporary variables**
 4. **Interchange of two independent adjacent statements.**

(i) Common sub-expression elimination

- Common sub expressions need not be computed over and over again. Instead they can be computed once and kept in store from where it's referenced when encountered again.

Example

$a := b + c$
$b := a - d$
$c := b + c$
$d := a - d$

The 2nd and 4th statements
 compute the same expression
 $b+c$ and $a-d$.

Hence the Basic block can be transformed to

$a := b + c$
$b := a - d$
$c := a$
$d := b$

(ii) Dead code elimination

- It's possible that a large amount of dead (useless) code may exist in the program.
- This might be especially caused when introducing variables and procedures as part of construction or error-correction of a program – once declared and defined, one forgets to remove them in case they serve no purpose.
- Eliminating these will definitely optimize the code.

(iii) Renaming of temporary variables

- A statement $t = b + c$ where t is a temporary name can be changed to $u = b + c$ where u is another temporary name, and change all uses of t to u .
- In this we can transform a basic block to its equivalent block called normal-form block.

(iv) Interchange of two independent adjacent statements

Two statements

$t1 := b + c$

t2 := x + y

can be interchanged or reordered in its computation in the basic block when value of t1 does not affect the value of t2.

(v) Algebraic Transformations

- Algebraic identities represent another important class of optimizations on basic blocks. This includes simplifying expressions or replacing expensive operation by cheaper ones.
- Another class of related optimizations is constant folding. Here we evaluate constant expressions at compile time and replace the constant expressions by their values. Thus the expression $2 * 3.14$ would be replaced by 6.28.
- Associative laws may also be applied to expose common sub expressions.

Example

- $x := x + 0$ or $x = x * 1$ can be removed.
- $x := y ** 2$ can be replaced by a cheaper statement $x := y * y$
- The compiler writer should examine the language carefully to determine what rearrangement of computations are permitted, since computer arithmetic does not always obey the algebraic identities of mathematics. Thus, a compiler may evaluate $x * y - x * z$ as $x * (y - z)$ but it may not evaluate $a + (b - c)$ as $(a + b) - c$.

3. LOOP OPTIMIZATIONS

- The loops where programs tend to spend the bulk of their execution time. The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.
- Three techniques are important for loop optimization
 1. **Code motion**, which moves code outside a loop;
 2. **Induction-variable elimination**, which we apply to replace variables from inner loop.
 3. **Reduction in strength**, which replaces and expensive operation by a cheaper one, such as a multiplication by an addition.

(i) Code Motion

- An important modification that decreases the amount of code in a loop is code motion.
- This transformation takes an expression that yields the same result independent of the number of times a loop is executed (a loop-invariant computation) and places the expression before the loop.
- Note that the notion “before the loop” assumes the existence of an entry for the loop.
- For example, evaluation of limit-2 is a loop-invariant computation in the following while statement:

```
while (i <= limit-2) /* statement does not change limit*/
```

Code motion will result in the equivalent of

```
t= limit-2;
```

```
while (i<=t) /* statement does not change limit or t */
```

(ii) Induction Variables

- Loops are usually processed inside out.
- When there are two or more induction variables in a loop, it may be possible to get rid of all but one, by the process of induction-variable elimination.

Example:

As the relationship $t4 := 4*j$ surely holds after such an assignment to $t4$ in Fig. and $t4$ is not changed elsewhere in the inner loop around B3, it follows that just after the statement $j := j - 1$ the relationship $t4 := 4*j - 4$ must hold. We may therefore replace the assignment $t4 := 4*j$ by $t4 := t4 - 4$. The only problem is that $t4$ does not have a value when we enter block B3 for the first time. Since we must maintain the relationship $t4 := 4*j$ on entry to the block B3, we place an initialization of $t4$ at the end of the block where j itself is initialized, shown by the dashed addition to block B1 in second Fig. The replacement of a multiplication by a subtraction will speed up the object code if multiplication takes more time than addition or subtraction, as is the case on many machines.

(iii) Reduction In Strength

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine.
- Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators.

For example,

1. x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine.
2. Fixed-point multiplication or division by a power of two is cheaper to implement as a shift. Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper.

(iv) Loop Unrolling

- The loop exit checks cost more CPU time. Loop unrolling tries to get rid of the checks completely or to reduce the number of checks.
- If you know a loop is only performed a certain number of times, or if you know the number of times it will be repeated is a multiple of a constant you can unroll this loop.

Example:

```
// old loop
for(int i=0; i<3; i++) {
    color_map[n+i] = i;
}
```

```
// unrolled version
int i = 0;
colormap[n+i] = i;
i++;
colormap[n+i] = i;
i++;
colormap[n+i] = i;
```

III. SYMBOL TABLE MANAGEMENT

Symbol table is an important data structure created and maintained by compilers in order to store information about the occurrence of various entities such as variable names, function names, objects, classes, interfaces, etc. Symbol table is used by both the analysis and the synthesis parts of a compiler.

A symbol table may serve the following purposes depending upon the language in hand:

- To store the names of all entities in a structured form at one place.
- To verify if a variable has been declared.
- To implement type checking, by verifying assignments and expressions in the source code are semantically correct.
- To determine the scope of a name (scope resolution).

4.1 SCOPE MANAGEMENT

- A compiler maintains two types of symbol tables: a **global symbol table** which can be accessed by all the procedures and **scope symbol tables** that are created for each scope in the program.
- The global symbol table contains names for one global variable (int value) and two procedure names, which should be available to all the child nodes shown above. The names mentioned in the pro_one symbol table (and all its child tables) are not available for pro_two symbols and its child tables.

This symbol table data structure hierarchy is stored in the semantic analyzer and whenever a name needs to be searched in a symbol table, it is searched using the following algorithm:

- First a symbol will be searched in the current scope, i.e. current symbol table.
- if a name is found, then search is completed, else it will be searched in the parent symbol table until,
- Either the name is found or global symbol table has been searched for the name.

Example:

To determine the scope of a name, symbol tables are arranged in hierarchical structure as shown in the example below:

```

...
int value=10;

void pro_one()
{
  int one_1;
  int one_2;

  {
    \
    int one_3;  |__ inner scope 1
    int one_4;  |
  }

```

```

    }      /

int one_5;

{      \
int one_6;  |_ inner scope 2
int one_7;  |
}      /

}

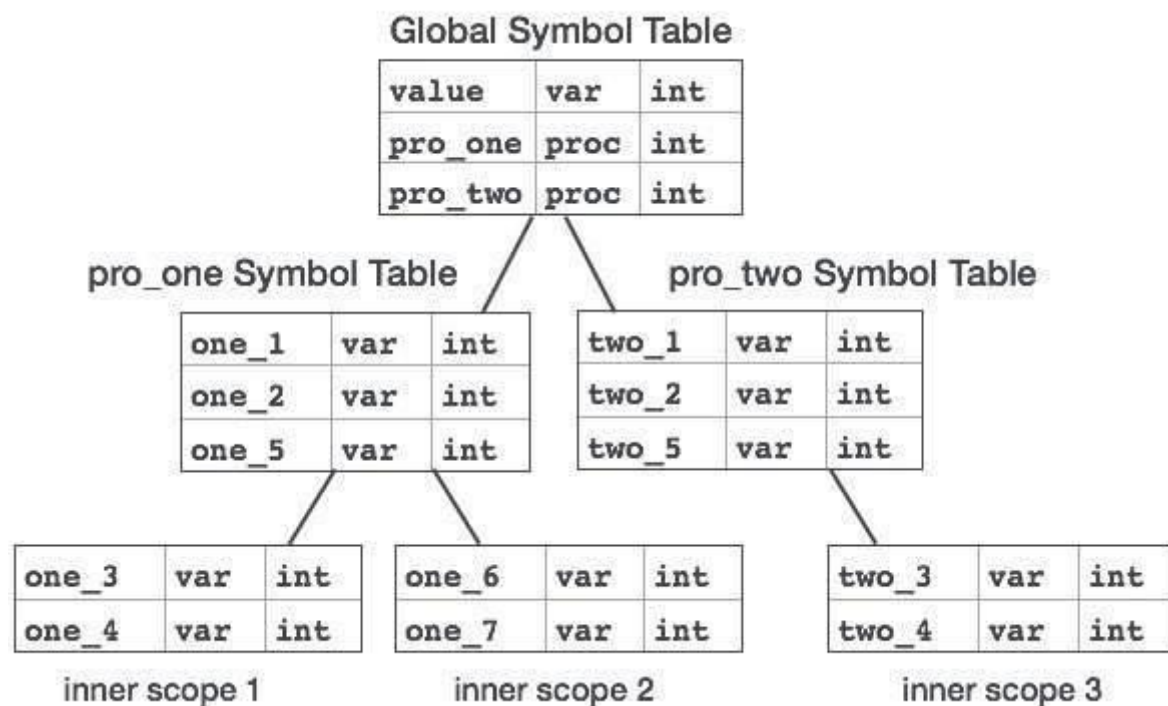
void pro_two()
{
int two_1;
int two_2;

{      \
int two_3;  |_ inner scope 3
int two_4;  |
}      /

int two_5;
}
...

```

The above program can be represented in a hierarchical structure of symbol tables:



4.2 The Role of a Symbol Table in Compilers

- A symbol table is a data structure, where information about program objects is gathered
- It is used in both the analysis and synthesis phases.
- The symbol table is **built up during the lexical and syntactic** analysis.
- Support for other phases during compilation:
- **Semantic analysis:** type conflict?
- **Code generation:** how much and what type of *run-time* space is to be allocated?
- **Error handling:** Has the error message already been issued?

4.3 Symbol Table Management

- Symbol table phase or symbol table management refer to the symbol table's storage structure, its construction in the analysis phase and its use during the whole compilation.
- It maps names into declarations such as mapping the variable name x to its type. More specifically, a symbol table stores
 1. For each type name, its type definition.
 2. For each variable name, its type. If the variable is an array, it also stores dimension information. It may also store storage class, offset in activation record etc.
 3. For each constant name, its type and value.
 4. For each function and procedure, its formal parameter list and its output type. Each formal parameter must have name, type, type of passing (by-reference or by-value), etc.

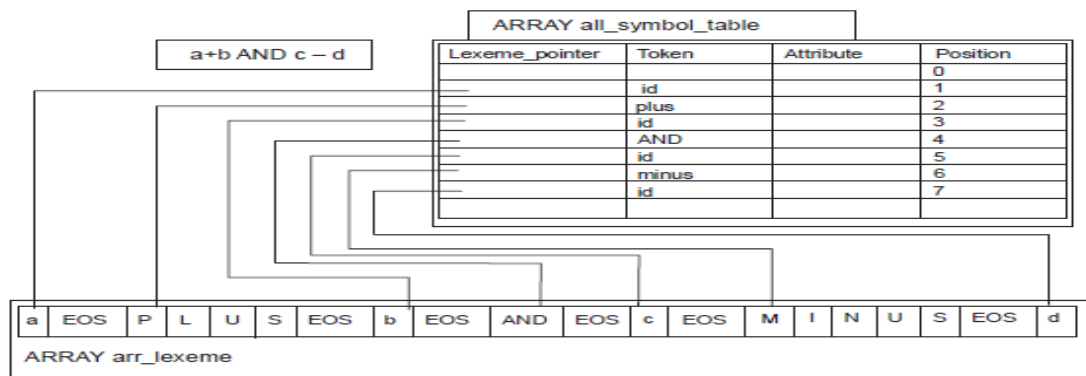
4.4 Operations on Symbol Table

The following are the operations on a symbol table,

1. **Insert (s,t)** – return index of new entry for string 's' and token 't'
2. **Lookup (s)** – return index of new entry for string 's' or 0 if 's' is not found.
3. **Scoping ()** – maintaining information about the scope and lifetime of a names.
4. **Handling Keywords:** Symbol table also handle reserve keywords like main, etc.

4.5 Symbol Table Implementation

- A separate array holds the character string forming an identifier. The string is terminated by an end-of-string character, denoted by EOS that may not appear in identifiers.
- Each entry in symbol-table is a record consists of many attributes of the symbol as fields.
- An entry for the identifier is created using insert (). After the insertion is made; 'n' is the index of the symbol-table entry for the string is communicated to the parser by setting tokenval to n, and the token in the token field of the entry is returned.



4.6 Data Structure used for Symbol Table Management

1. Linear List
2. Trees
3. Hash table

1. Linear List

- The simplest and easiest to implement data structure for symbol table is a linear list of records. A single array or collections of several arrays are used for this purpose to store name and their associated information.
- The names are added to end of array. End of array always marks by a point known as space.
- To Insert () any name in the list then
 - Searching is done in whole array from 'space' to beginning of array.
 - If word is not found in array then create an entry at 'space' and increment 'space' by one or value of data type.
- In implementation of symbol table first field always empty because when 'object-lookup' work then it will return '0' to indicate no string in symbol table.

Example:

```

int a,b,c;
int sum (int x, int y)
{
    a = x+y
    return (a)
}
main ()
{
    int u,
    u=sum (5,6);
}
  
```

→

→

→

→

→

Variable	Information	Space(byte)
u	Integer	2 byte
a	Integer	2 byte
b	Integer	2 byte
c	Integer	2 byte
x	Integer	2 byte
y	Integer	2 byte
sum	Integer	2 byte

← Space

Complexity: If any symbol table has 'n' names then for inserting any new name we must search list 'n' times in worst case. So cost of searching is $O(n)$ and if we want to insert 'n' name then cost of this insert is $O(n^2)$ in worst case.

Self Organizing List

- To reduce the time of searching it can add an additional field 'linker' to each record field or each array index.
- When a name is inserted then it will insert at 'space' and manage all linkers to other existing name.

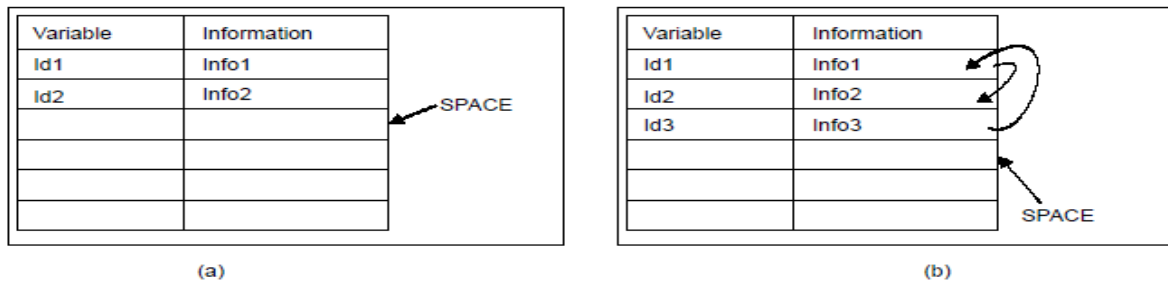


Figure 9.3: Symbol table as self organizing list

In above figure (a) represent the simple list and (b) represent self organizing list in which Id1 is related to Id2 and Id3 is related to Id1.

2. Tree

- Another approach to organize a symbol table is that adds two additional link fields i.e. left and right child, as binary search tree. All names are created as child of root node that always follows the property of binary tree.
- For inserting any name it always follow binary search tree insert algorithm.
- Each subprogram has a symbol table associated to its node in the abstract syntax tree.
- The main program has a similar table for globally declared objects.
- Quicker than linear lists and also easy to represent scoping.

Example:

```

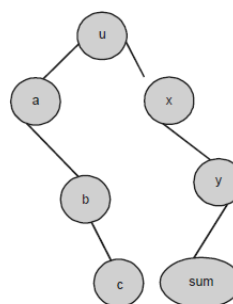
int a,b,c;
int sum (int x, int y)
{
    a = x+y
    return (a)
}
main ()
{
    int u,
    u=sum (5,6);
}

```

→

→

→

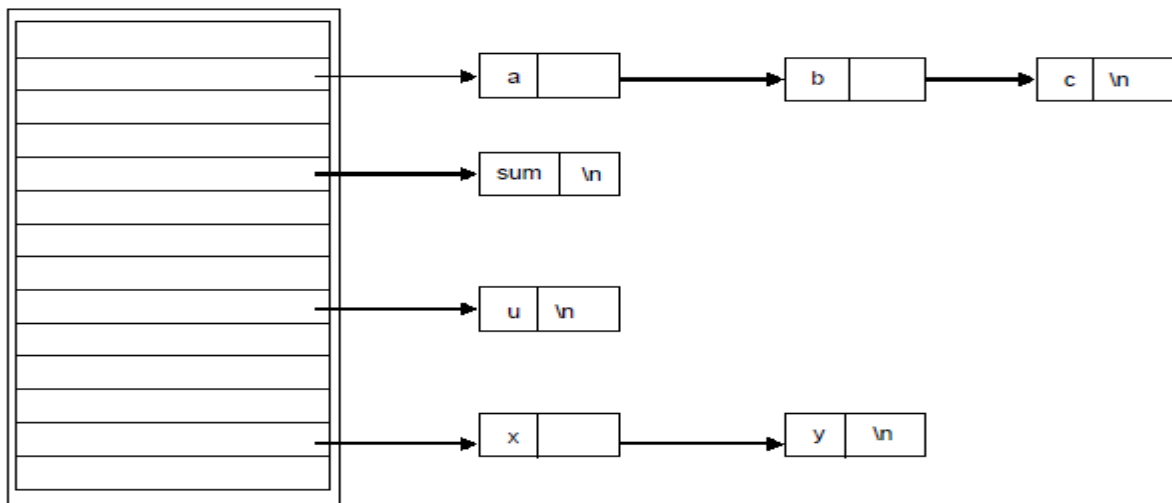


3. Hash Table

- A **hash table**, or a **hash map**, is a data structure that associates keys with values 'Open hashing' is a key that is applied to hash table.
- In hashing –open, there is a property that no limit on number of entries that can be made in table. Hash table consist an array and several buckets attached to array according to hash function.

- Main advantage of hash table is that we can insert or delete any number or name in $O(n)$ time, if data are search linearly and there are 'n' memory location where data is stored.
- Using hash function any name can be search in $O(1)$ time. However, the rare worst-case lookup time can be as bad as $O(n)$.
- A good hash function is essential for good hash table performance. A poor choice of a hash function is likely to lead to clustering, in which probability of keys mapping to the same hash bucket (i.e. a collision) occur. One organization of a hash table that resolves conflicts is chaining.

Example:



Pros and Cons

- Very quick search
- Relatively complicated
- Extra space required k words for the hash table.

More difficult to introduce scoping.